

Post-QKV ReLU Placement in Pythia-14M: Quality, Exact Zeros, and Logical Compute

Thiago Mattar
thiagocmattar@gmail.com

2026-07-17

- **Best tested post-QKV no-pressure placement.** PRE-RoPE Q/K gates plus a V gate raise model-wide zero-product opportunity from 5.90% for Three-ReLU AdamW to 11.53%, for only +0.0156 validation loss.
- **Best tested post-QKV pressured tradeoff.** POST-RoPE OR reaches 16.48% model-wide opportunity for +0.0366 loss versus its AdamW architecture control. POST-RoPE OL1 reaches 17.82% for +0.0740.
- **Largest sparsity is not the best tradeoff.** PRE-RoPE OR and OL1 reach about 22.7% model-wide opportunity, but cost +0.486 and +0.524 validation loss versus PRE AdamW.
- **Clipping headroom is concentrated in POST.** Among the tested thresholds and under a shared +0.05 within-sweep validation-loss budget, POST gains 1.12–1.20 percentage points of R_{model} ; every other tested architecture/method gains at most 0.60 points.
- **RoPE matters.** PRE gate zeros are not automatically QK zeros: RoPE reduces AdamW Q/K zero rates from 8.85/15.12% at the gates to 6.83/12.78% at the actual QK operands.
- **Execution remains dense.** All percentages are exact logical zero products. Dense PyTorch does not skip them, so observed tokens/s is not a sparse-kernel speedup.
- **Recommendation.** Keep PRE AdamW and POST OR as the next controls; tune cost-weighted Q/K/V pressure, test pair-aware PRE gates, and benchmark only after a zero-aware kernel exists.

1 Question and Matched Design

The test asks where post-QKV ReLUs should sit and whether OR or OL1 can exploit them without erasing attention quality. Every run is random-initialized Pythia-14M pretraining over the same MiniPile token-cache pass: 22,762 optimizer steps, 1,491,730,432 training tokens, seed 0, bf16, one RTX 5070 Ti Laptop GPU, and full validation on 692,224 tokens.

The control ladder is:

- **Stock:** GELU only in the MLP hidden path.
- **One-ReLU:** ReLU only at `mlp_hiddens`.
- **Three-ReLU:** ReLU at `attention_inputs`, `mlp_inputs`, and `mlp_hiddens`.
- **Six-ReLU PRE:** Three-ReLU plus separate Q/K ReLUs before RoPE and a V ReLU before PV.
- **Six-ReLU POST:** Three-ReLU plus separate Q/K ReLUs after RoPE and a V ReLU before PV.

The final model LayerNorm is unchanged in every case.

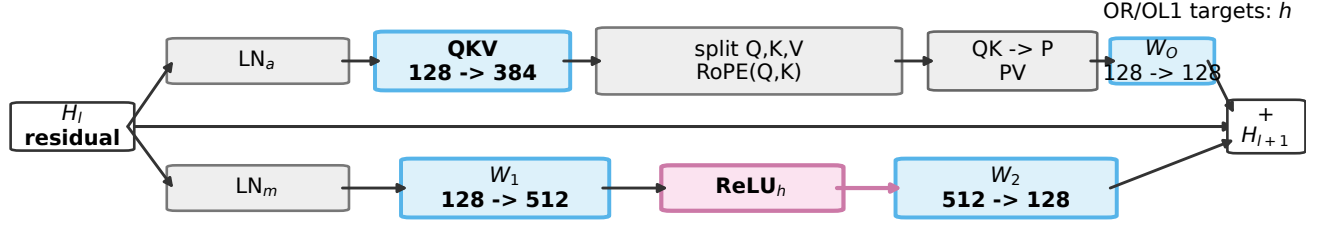
The figure generator pins exact run IDs in `REPORT05.PINNED.RUN_IDS` rather than selecting newer siblings. Config 50 is clean; included legacy configs 77, 79, 81, 98, 99, and 103 have coherent artifact envelopes but record `git_dirty: true`. Configs 107–112 were launched from clean commit `f00a5cd`.

OR is orthogonal Ricker pressure; OL1 is orthogonal L1 pressure. They target only the gates introduced by each architecture: h for One-ReLU; a, m, h for Three-ReLU; and Q/K/V for PRE and POST. The original three gates remain active but unpressured in both Six-ReLU families. All architectures contain 14,067,712 trainable parameters; ReLUs and these pressure rules add no parameters.

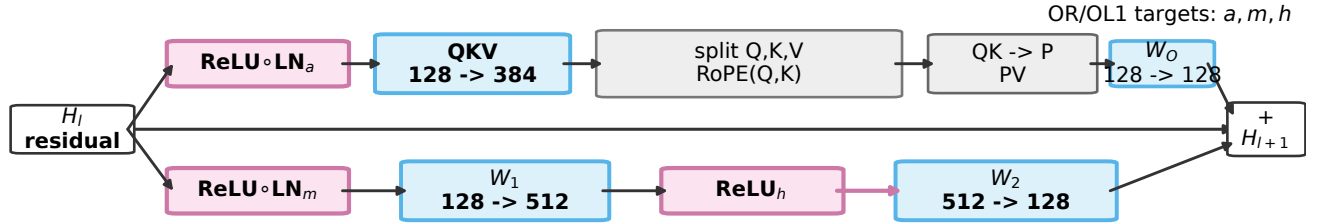
Pythia-14M ReLU Architecture Ladder

Parallel residual in every case: $H_{l+1} = H_l + O_l + M_l$; $d = 128$, 4 heads, $d_h = 32$, partial RoPE on 8 coordinates

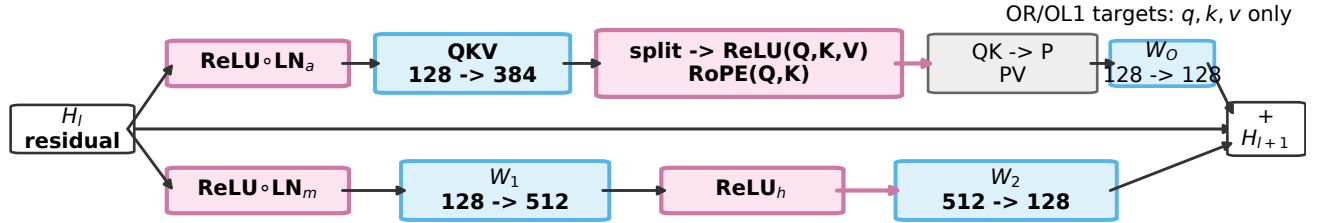
(a) One-ReLU (MLP hidden)



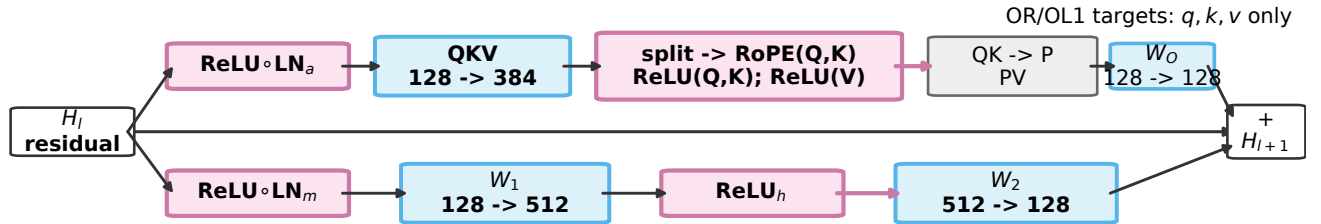
(b) Three-ReLU



(c) Six-ReLU PRE



(d) Six-ReLU POST



Gate aliases: a = attention_inputs, m = mlp_inputs, h = mlp_hiddens, $q/k/v$ = post-QKV gate outputs.

Magenta boxes create exact zeros; blue boxes are learned projections.
Stock AdamW replaces the hidden ReLU with GELU and has no other gates.

Figure 1: One Pythia-14M parallel-residual block for each ReLU architecture. QKV maps $128 \rightarrow 384$, W_1 maps $128 \rightarrow 512$, and W_2 maps $512 \rightarrow 128$. Magenta gates create exact zeros; blue boxes are learned projections.

For one block, with $d = 128$, four heads, and head width $d_h = 32$,

$$\begin{aligned}
a_l &= \text{ReLU}(\text{LN}_l^{\text{att}}(H_l)), & [Q_l^0, K_l^0, V_l^0] &= a_l W_{\text{qkv},l} + b_{\text{qkv},l}, \\
S_l &= Q_l K_l^\top / \sqrt{d_h}, & P_l &= \text{softmax}(\text{causal_mask}(S_l)), \\
C_l &= P_l V_l, & O_l &= C_l W_{o,l} + b_{o,l}, \\
m_l &= \text{ReLU}(\text{LN}_l^{\text{mlp}}(H_l)), & h_l &= \text{ReLU}(m_l W_{1,l} + b_{1,l}), \\
M_l &= h_l W_{2,l} + b_{2,l}, & H_{l+1} &= H_l + O_l + M_l.
\end{aligned}$$

One-ReLU omits the first two displayed ReLUs; stock also replaces the hidden ReLU with GELU. PRE uses $Q = \text{RoPE}(\text{ReLU}(Q^0))$ and $K = \text{RoPE}(\text{ReLU}(K^0))$. POST uses $Q = \text{ReLU}(\text{RoPE}(Q^0))$ and likewise for K . Both use $V = \text{ReLU}(V^0)$. Plain ReLU is not useful after softmax because valid P entries are positive; learnable probability thresholding remains a separate TODO.

For $T = 2,048$, $Q, K, V \in \mathbb{R}^{T \times 4 \times 32}$, $S, P \in \mathbb{R}^{4 \times T \times T}$, and the concatenated context $C \in \mathbb{R}^{T \times 128}$.

2 Training Endpoints

Table 1: Matched one-pass endpoints. The Parameters column lists nondefault pressure settings and sites: w is pressure weight, c is the Ricker zero-crossing radius/shape parameter, σ is its Gaussian-envelope width, b is the Adam-step budget, and a, m, h are attention input, MLP input, and MLP hidden. Every row has 14,067,712 trainable parameters.

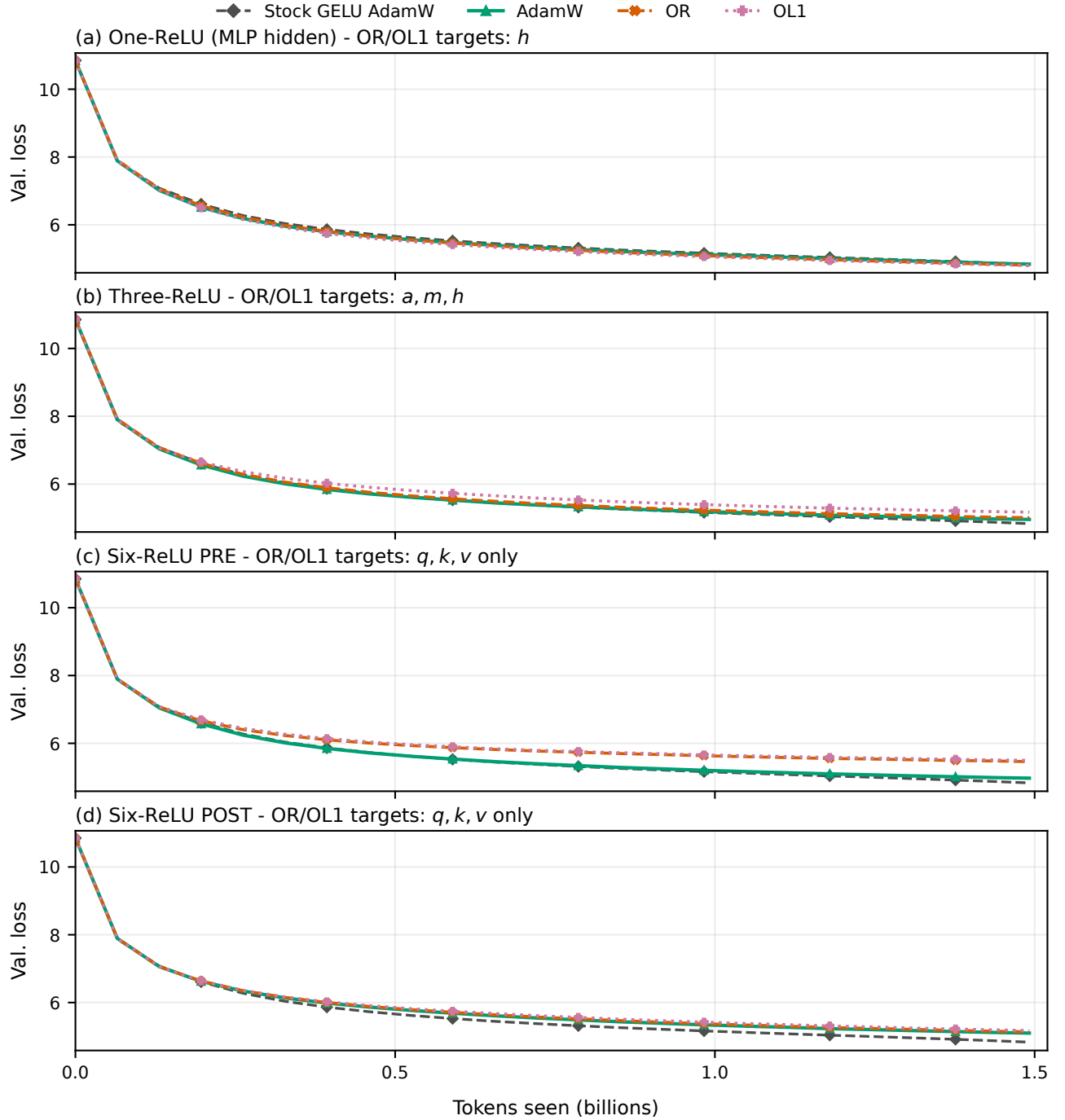
Config	Architecture	Name	Parameters	Val. loss	Time (h)	Tokens/s
50	Stock (GELU)	AdamW	—	4.8317	3.010	137,648
77	One-ReLU	AdamW	—	4.8404	2.897	143,021
79	One-ReLU	OR	$w = 1, c = \sigma = .05, b = .5; h$	4.8087	4.018	103,131
81	One-ReLU	OL1	$w = 5, b = .5; h$	4.7981	3.788	109,379
98	Three-ReLU	AdamW	—	4.9564	3.449	120,139
103	Three-ReLU	OR	$w = 1, c = \sigma = .05, b = .5; a, m, h$	5.0091	4.404	94,086
99	Three-ReLU	OL1	$w = 5, b = .5; a, m, h$	5.1706	4.318	95,971
107	Six-ReLU PRE	AdamW	—	4.9720	3.389	122,279
108	Six-ReLU PRE	OR	$w = 1, c = \sigma = .05, b = .5; Q, K, V$	5.4577	4.477	92,566
109	Six-ReLU PRE	OL1	$w = 5, b = .5; Q, K, V$	5.4962	4.035	102,701
110	Six-ReLU POST	AdamW	—	5.0991	3.455	119,932
111	Six-ReLU POST	OR	$w = 1, c = \sigma = .05, b = .5; Q, K, V$	5.1358	4.212	98,389
112	Six-ReLU POST	OL1	$w = 5, b = .5; Q, K, V$	5.1731	4.087	101,399

The AdamW architecture comparison isolates placement. One-ReLU is essentially loss-neutral versus stock (+0.0087). Three-ReLU costs +0.1247. Adding PRE gates to Three-ReLU costs only another +0.0156, whereas POST costs +0.1427. Within architectures, One-ReLU OR/OL1 improve the single-run endpoint; Three-ReLU OR has a modest +0.0527 cost; PRE pressure is expensive; POST OR remains close to its control.

Training throughput follows dense execution and method overhead. One-ReLU AdamW happens to be faster than stock, while additional gates and the extra pressure-gradient work reduce throughput elsewhere. PyTorch does not automatically turn scalar ReLU zeros into skipped dense matmul work, so these timings should not be interpreted as sparsity speedups.

Validation Learning Curves Across the ReLU Architecture Ladder

One seed per run; fixed 22,762-step / 1.492B-token budget; 692,224 validation tokens per point



Stock GELU AdamW is repeated in every panel as the common reference.
 Curves are single-run observations; no uncertainty bands.

Figure 2: Full-validation learning curves. Stock GELU AdamW is repeated as the common reference; every point evaluates the complete 692,224-token validation set. These are single runs without uncertainty bands.

3 Exact Zeros and Logical Products

An activation coordinate is exactly zero only when its computed floating-point value satisfies $x = 0$; no tolerance or histogram bin is used. Counts pool all 338 complete sequences, all six layers, and all feature/head coordinates. The final 1,444 cache tokens that do not form a complete 2,048-token block are excluded.

For operation $o \in \{\text{QKV}, \text{QK}, \text{PV}, W_o, W_1, W_2\}$, a logical product is zero when its activation operand is zero; for QK and PV, either operand may be zero. Only valid causal QK/PV pairs are counted. Define

$$R_{\text{block}} = \frac{\sum_{l,o} Z_{l,o}^{\text{prod}}}{\sum_{l,o} N_{l,o}^{\text{prod}}},$$

$$R_{\text{model}} = \frac{\sum_{l,o} Z_{l,o}^{\text{prod}}}{\sum_{l,o} N_{l,o}^{\text{prod}} + N_{\text{tok}} d |\mathcal{V}|}, \quad |\mathcal{V}| = 50,304.$$

The pooled block denominator is 1,905,886,494,720 products; the dense LM head adds 4,457,169,420,288. Thus $R_{\text{model}} = 0.299524 R_{\text{block}}$ here. No head sparsity is credited, and neither fraction is measured latency.

Table 2: Full-validation exact-zero outcomes. z_a, z_m, z_h are pooled rates at the attention-input, MLP-input, and MLP-hidden ReLU outputs; z_Q^g, z_K^g, z_V are post-QKV gate-output rates. An em dash means that gate does not exist. R_{block} and R_{model} use actual matmul operands, so PRE Q/K are counted after RoPE.

Cfg.	Architecture	Method	Val. loss	z_a	z_m	z_h	z_Q^g	z_K^g	z_V	R_{block}	R_{model}
50	Stock	AdamW	4.8317	—	—	—	—	—	—	0.00%	0.00%
77	One	AdamW	4.8404	—	—	52.06%	—	—	—	7.44%	2.23%
79	One	OR	4.8087	—	—	93.21%	—	—	—	13.31%	3.99%
81	One	OL1	4.7981	—	—	93.57%	—	—	—	13.36%	4.00%
98	Three	AdamW	4.9564	49.54%	50.14%	50.72%	—	—	—	19.71%	5.90%
103	Three	OR	5.0091	72.74%	71.00%	75.15%	—	—	—	28.66%	8.59%
99	Three	OL1	5.1706	83.07%	39.25%	48.87%	—	—	—	21.48%	6.43%
107	PRE	AdamW	4.9720	48.90%	50.09%	50.74%	8.85%	15.12%	47.77%	38.48%	11.53%
108	PRE	OR	5.4577	48.29%	49.75%	51.02%	90.62%	92.21%	94.32%	76.03%	22.77%
109	PRE	OL1	5.4962	47.50%	48.68%	51.17%	100.00%	100.00%	94.28%	75.73%	22.68%
110	POST	AdamW	5.0991	48.52%	50.06%	50.11%	15.68%	23.18%	49.24%	43.12%	12.91%
111	POST	OR	5.1358	48.56%	49.82%	50.19%	64.13%	65.77%	50.39%	55.03%	16.48%
112	POST	OL1	5.1731	48.54%	49.72%	50.37%	81.21%	81.39%	51.33%	59.50%	17.82%

The table separates sparsity from quality. PRE OR/OL1 nearly eliminate Q/K/V operands and maximize logical opportunity, but their loss increase is large. POST pressure produces less extreme attention sparsity and a substantially better quality tradeoff. One-ReLU pressure is inexpensive in quality and reaches 4.00%, near the W2-only theoretical R_{model} ceiling of 4.278%.

RoPE changes the PRE mask before QK:

PRE method	z_Q^g	z_Q^{QK}	z_K^g	z_K^{QK}
AdamW	8.85%	6.83%	15.12%	12.78%
OR	90.62%	88.36%	92.21%	90.58%
OL1	100.00%	100.00%	100.00%	100.00%

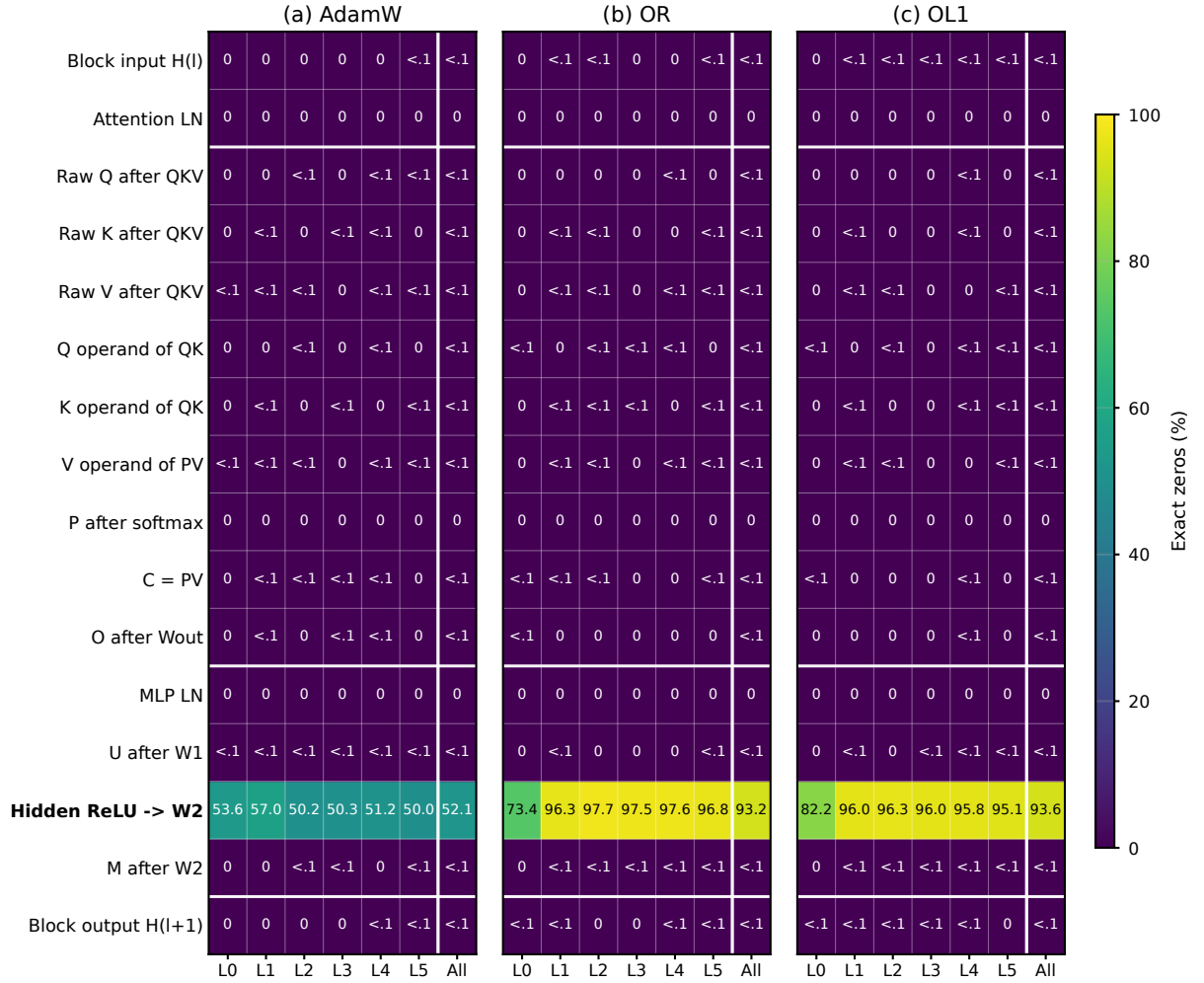
An isolated zero in a rotary pair can be repopulated by its paired coordinate. An all-zero pair stays zero; the non-rotary coordinates also pass through unchanged. POST gates operate on the final QK operands, so their gate and operand rates coincide.

4 Where Exact Zeros Persist

The following four figures use the same computation order. Each architecture is a 1×3 comparison of AdamW, OR, and OL1; columns L0–L5 are layer-specific and “All” pools integer counts. The compact labels make the densification boundaries visible without overflowing heatmap cells.

Where Exact Zeros Persist Through Pythia-14M Blocks: One-ReLU (MLP hidden)

Direct counts over 692,224 validation tokens; All = count-weighted pool over L0-L5



Exact zero: computed value == 0 (no tolerance). Each cell is zero values / produced values.

Figure 3: One-ReLU propagation. The hidden ReLU creates zeros immediately before W_2 ; the dense W_2 sum and parallel residual do not preserve that mask.

Where Exact Zeros Persist Through Pythia-14M Blocks: Three-ReLU

Direct counts over 692,224 validation tokens; All = count-weighted pool over L0-L5

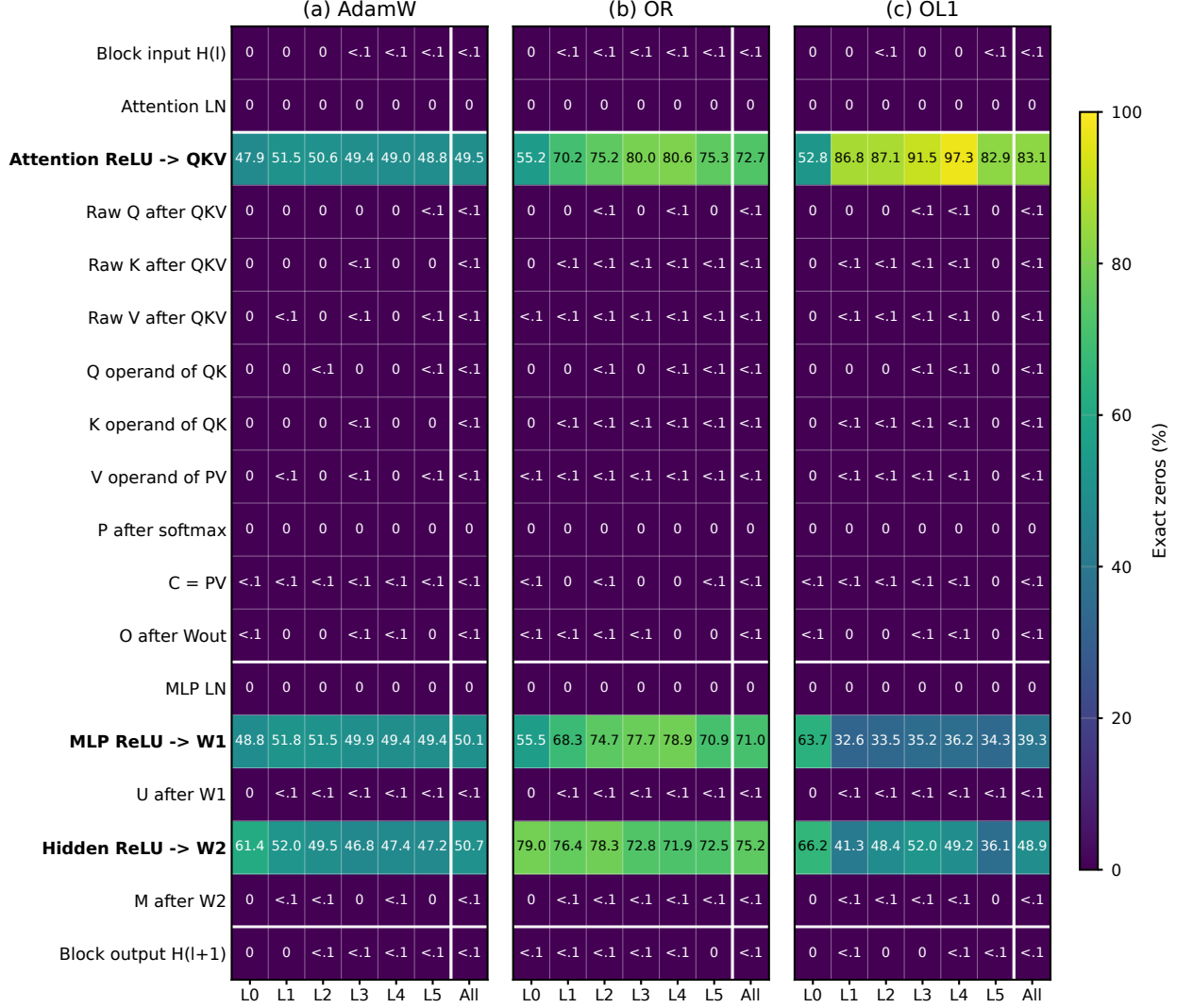
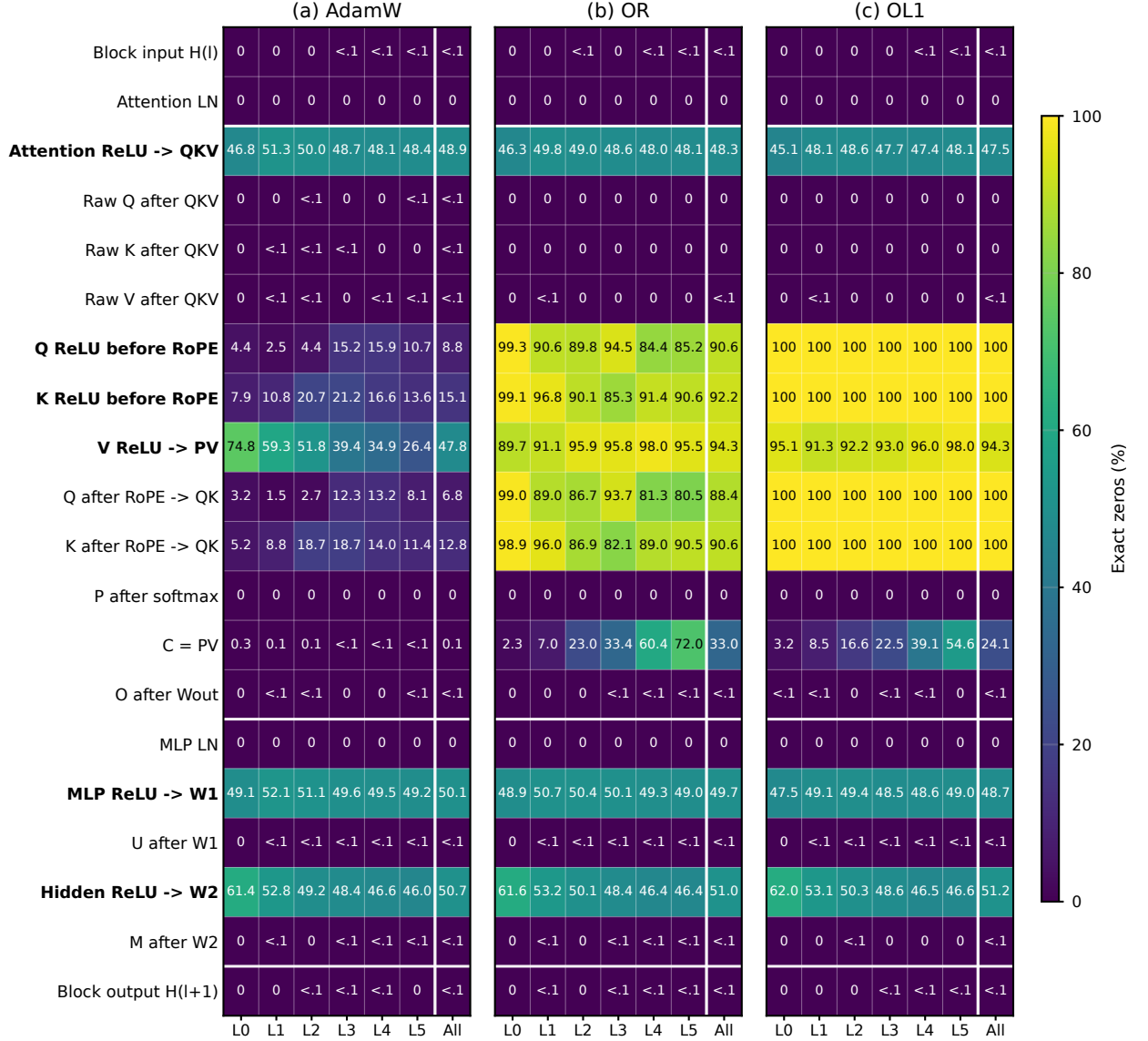


Figure 4: Three-ReLU propagation. Attention-input and MLP-input zeros expose QKV and W_1 products, but the corresponding projection outputs become dense; the hidden ReLU creates a separate W_2 mask.

Where Exact Zeros Persist Through Pythia-14M Blocks: Six-ReLU PRE (Q/K gate before RoPE)

Direct counts over 692,224 validation tokens; All = count-weighted pool over L0-L5



Exact zero: computed value == 0 (no tolerance). Each cell is zero values / produced values.

Figure 5: Six-ReLU PRE propagation. Q/K gates act before RoPE, so the gate and actual QK-operand rows differ. V zeros enter PV directly; when they cover every contributing value coordinate they also yield exact-zero context entries before W_o .

Where Exact Zeros Persist Through Pythia-14M Blocks: Six-ReLU POST (Q/K gate after RoPE)

Direct counts over 692,224 validation tokens; All = count-weighted pool over L0-L5

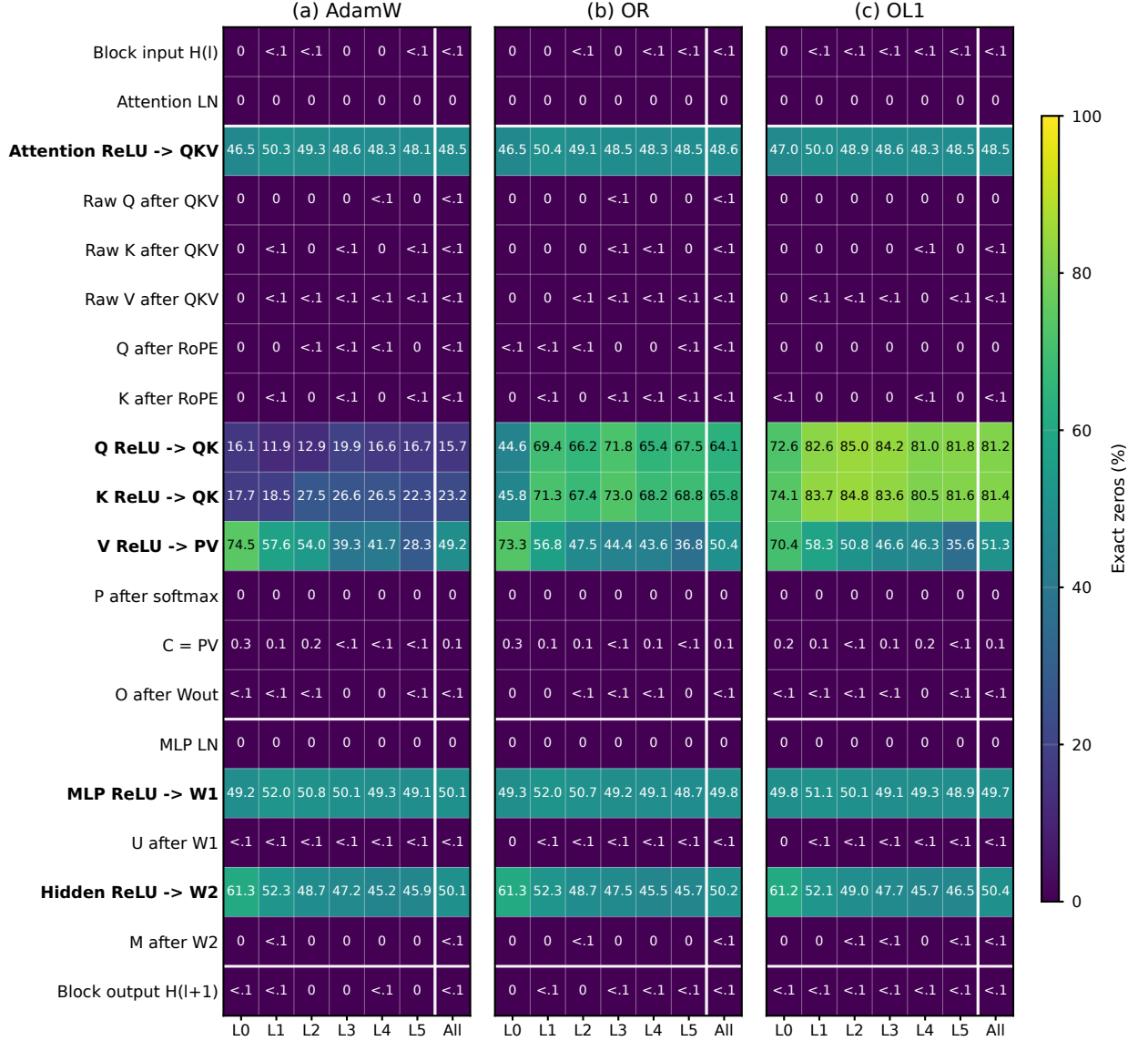


Figure 6: Six-ReLU POST propagation. Q/K gates directly define the QK operands; the V gate defines the sparse side of PV. Softmax P remains positive on valid positions, while context, output projection, and residual stream are effectively dense.

The mechanism is usually local. A zero before a linear projection removes one input–weight product for every output coordinate, but the output coordinate sums the remaining dense inputs. POST Q/K gates directly define QK operands; PRE gates do so only where their zeros survive RoPE. V gates expose PV products even though P stays positive. PV can also preserve substantial exact-zero context—33.02% for PRE OR and 24.08% for PRE OL1—before W_o makes its output effectively dense.

5 Activation and Weight Distributions

Exact-zero atoms are already reported in Table 2; the activation curves below show density conditional on $x \neq 0$. Weight curves pool the six final-checkpoint matrices. Q/K/V consumers have no single learned downstream matrix: Q and K enter QK, while V is multiplied by P .

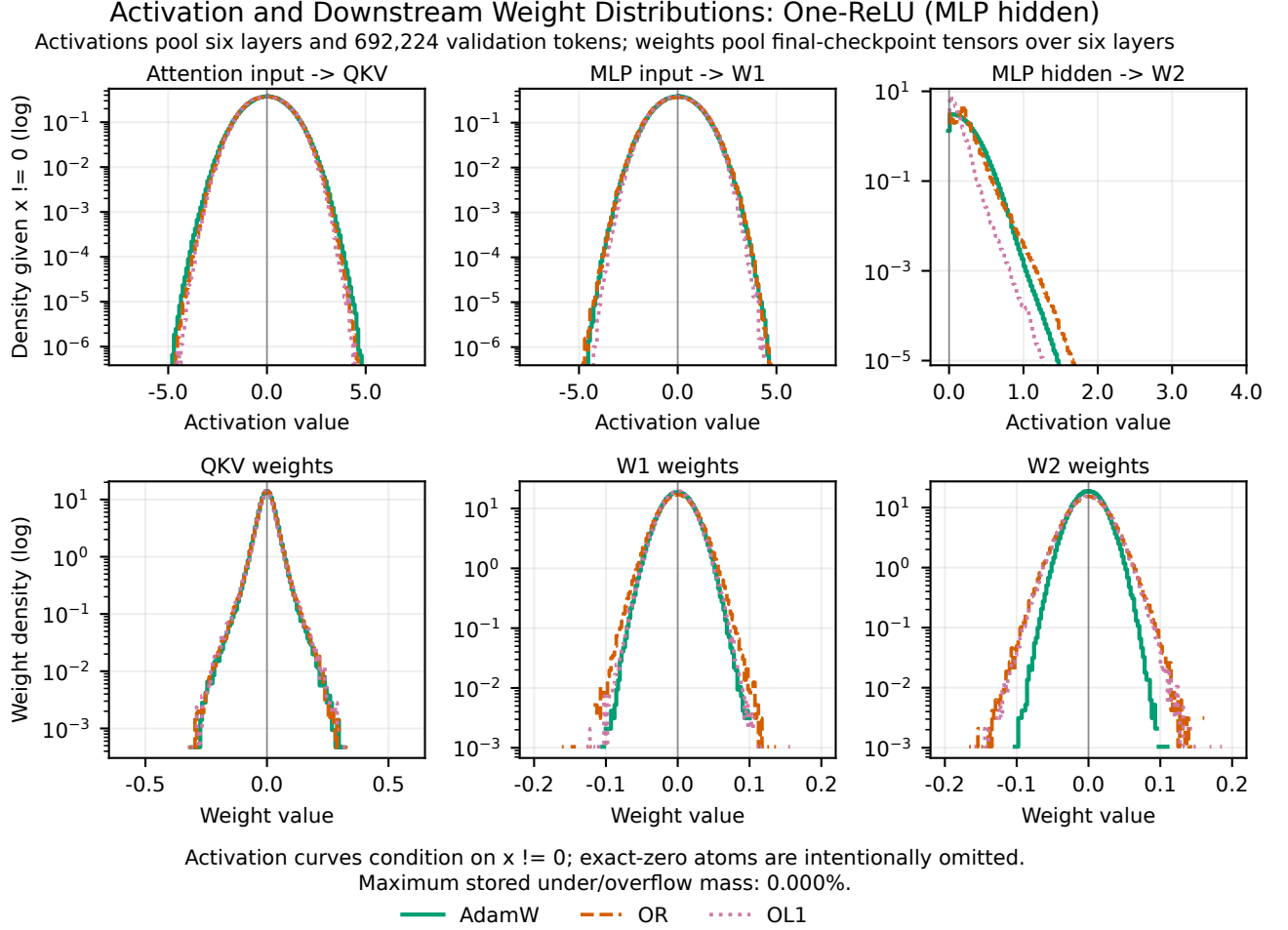


Figure 7: One-ReLU activation and downstream learned-weight densities. OR and OL1 strongly change the surviving MLP-hidden distribution while the learned matrices remain dense.

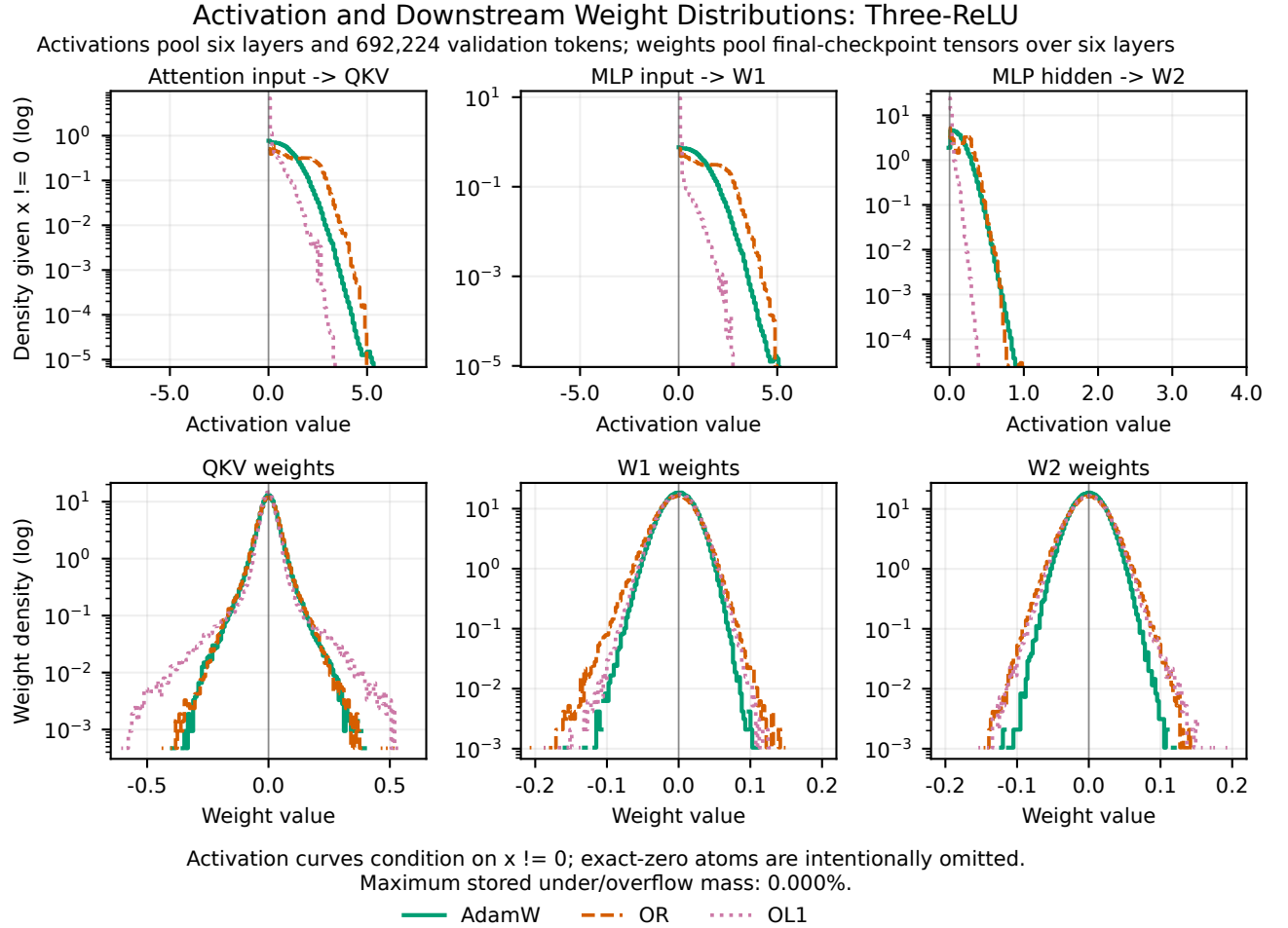


Figure 8: Three-ReLU activation and downstream learned-weight densities at the QKV, W_1 , and W_2 interfaces.

Activation and Downstream Weight Distributions: Six-ReLU PRE (Q/K gate before RoPE)

Activations pool six layers and 692,224 validation tokens; weights pool final-checkpoint tensors over six layers

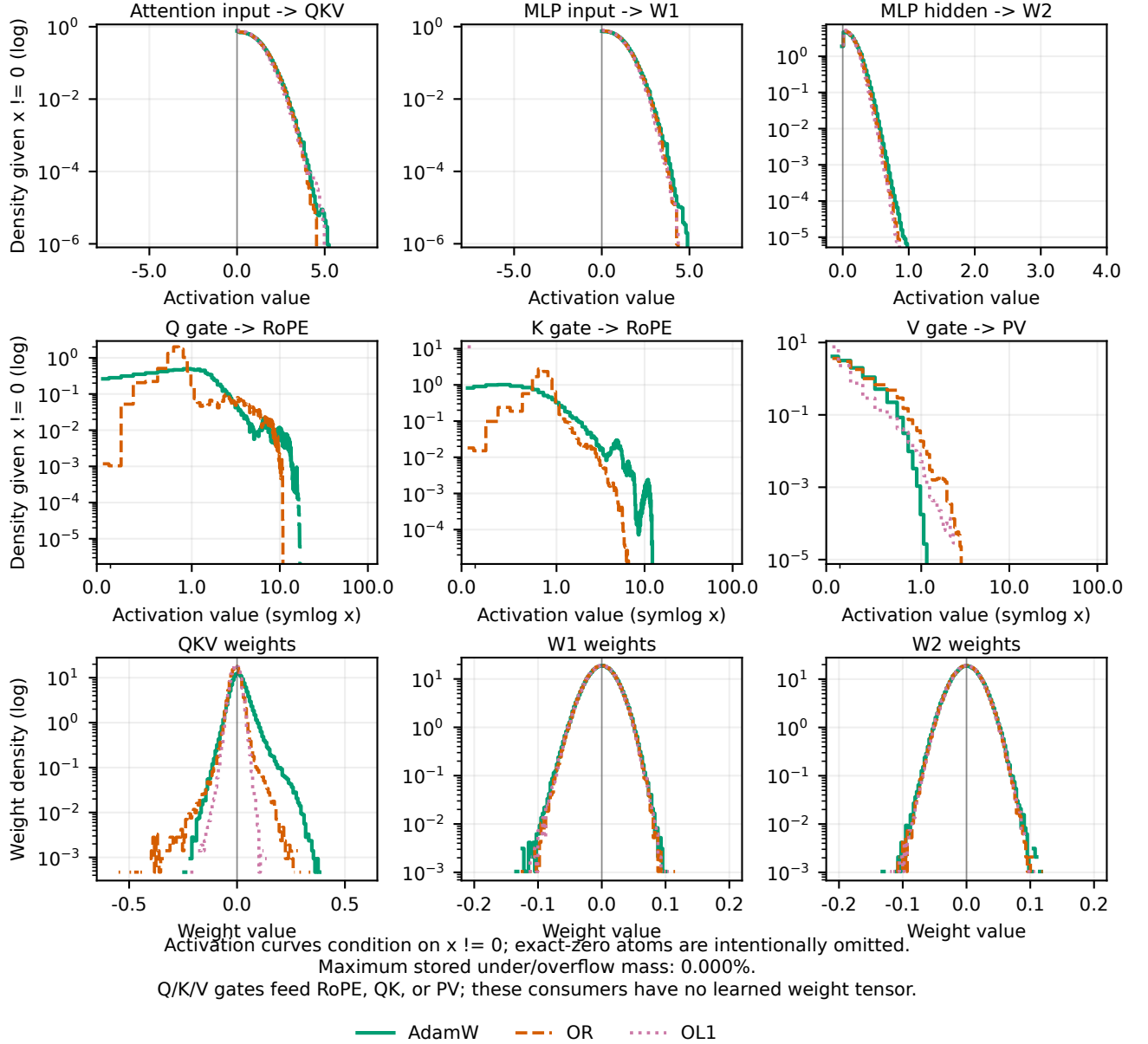


Figure 9: Six-ReLU PRE activation and weight densities. The Q/K/V panels show the nonzero tail at the new gates; exact-zero mass is intentionally not converted into a density spike. PRE OL1 leaves no nonzero Q values and only two nonzero K values, so its Q curve is absent and its K tail is not distributionally meaningful.

Activation and Downstream Weight Distributions: Six-ReLU POST (Q/K gate after RoPE)

Activations pool six layers and 692,224 validation tokens; weights pool final-checkpoint tensors over six layers

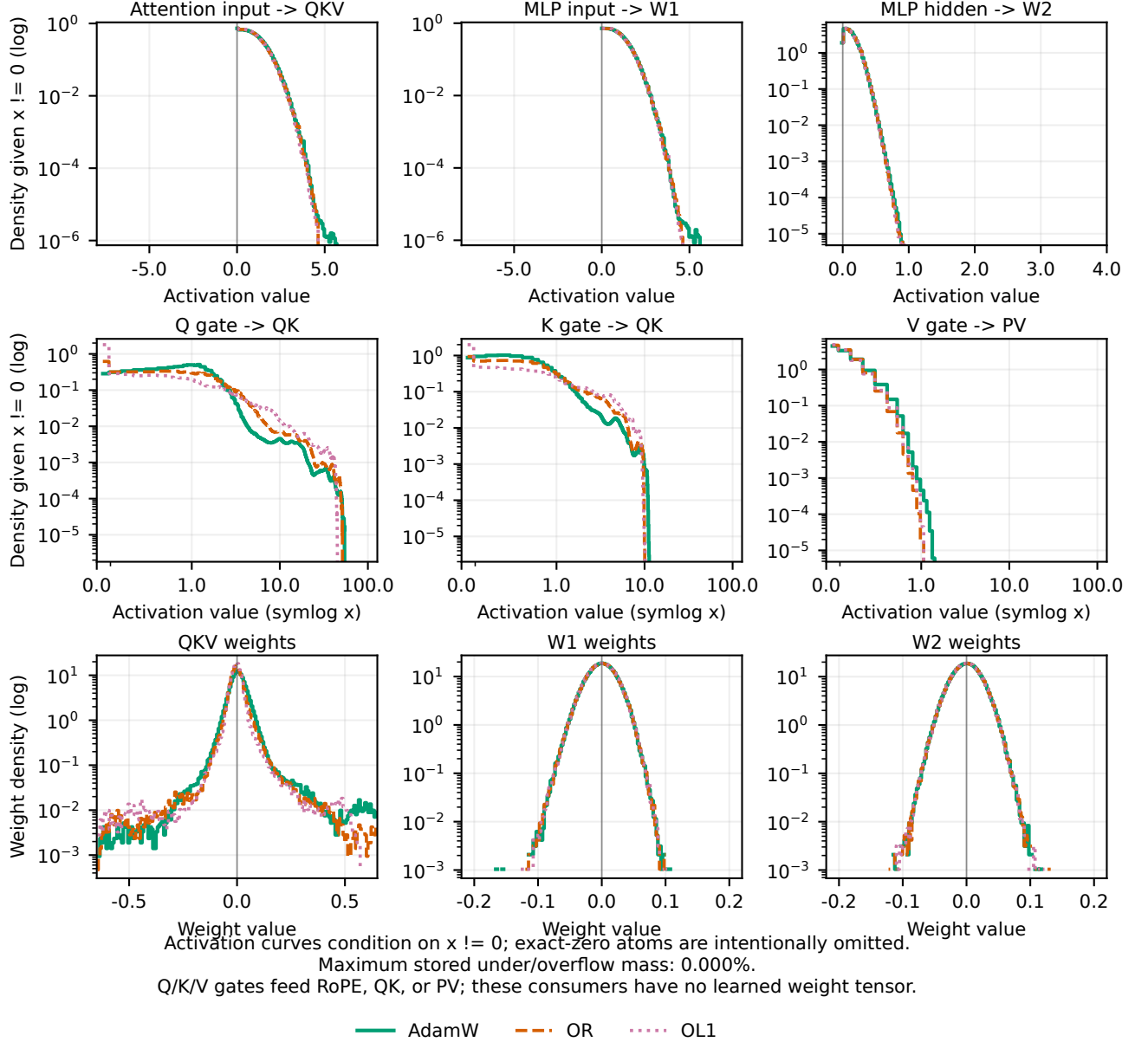


Figure 10: Six-ReLU POST activation and weight densities. Shared method identities make the placement-dependent change in the surviving Q/K tail directly comparable with PRE.

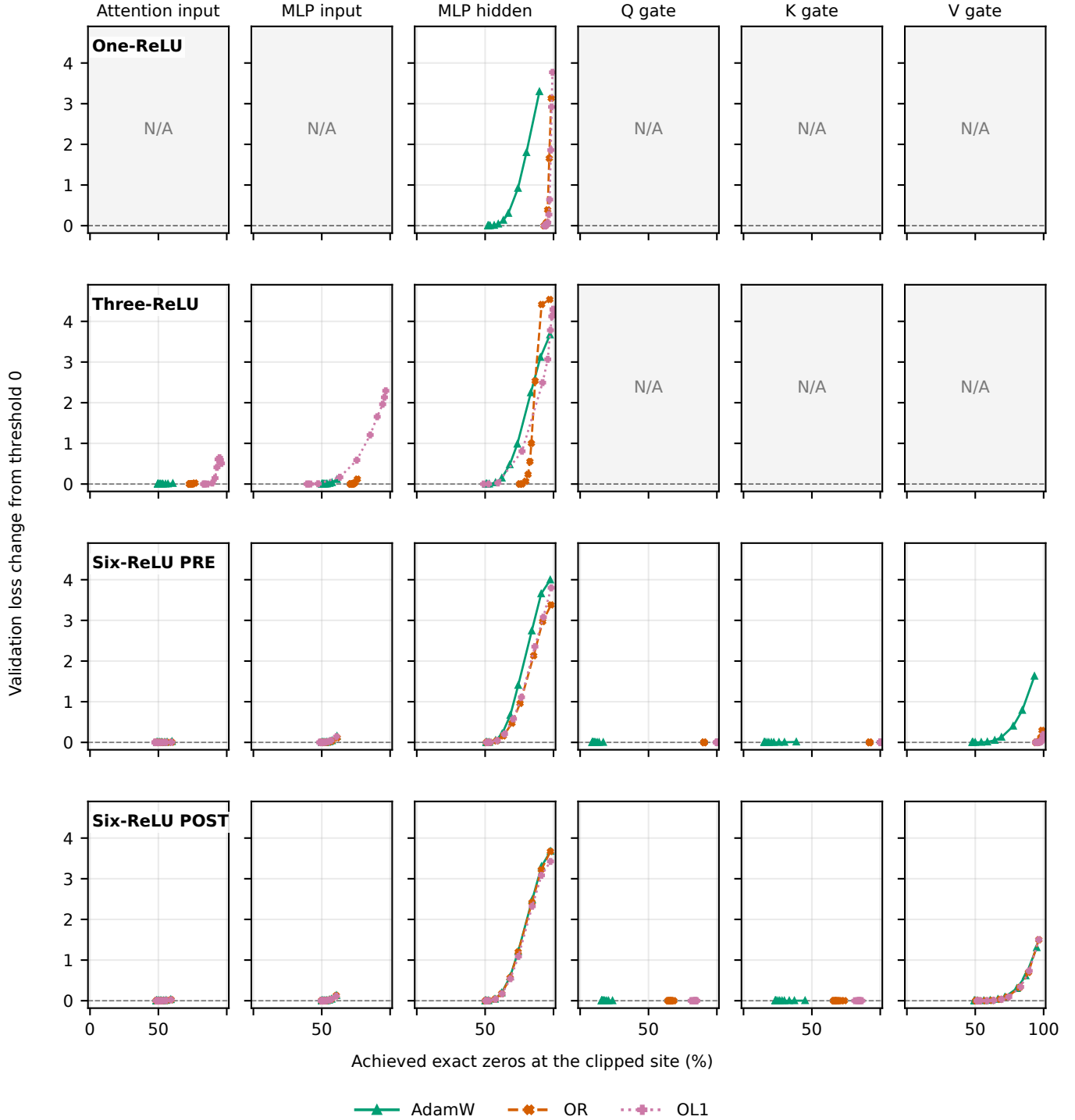
The distribution shift follows the pressure scope. Where nonzero Q/K/V values remain, their conditional distributions and QKV weights separate by method; W_1 and W_2 curves largely overlap, consistent with a, m, h remaining unpressured. One- and Three-ReLU pressure instead changes the surviving MLP/input distributions and their downstream matrices more visibly.

6 Post-Hoc Clipping Frontiers

Post-hoc clipping replaces values with zero when $|x| \leq t$ for $t \in \{0, .001, .003, .01, .03, .05, .075, .1, .15, .2, .3\}$. Every sweep evaluates the full validation cache. Loss change is measured from threshold zero within the same sweep; this is required because direct QK/PV counting forces eager attention. Site-specific curves isolate sensitivity. Joint curves clip every ReLU gate present in the architecture and count the resulting actual logical products.

Site-Specific Post-Hoc Clipping Frontiers

692,224 validation tokens per point; all panels share axes; loss change is relative to threshold 0 within the same sweep

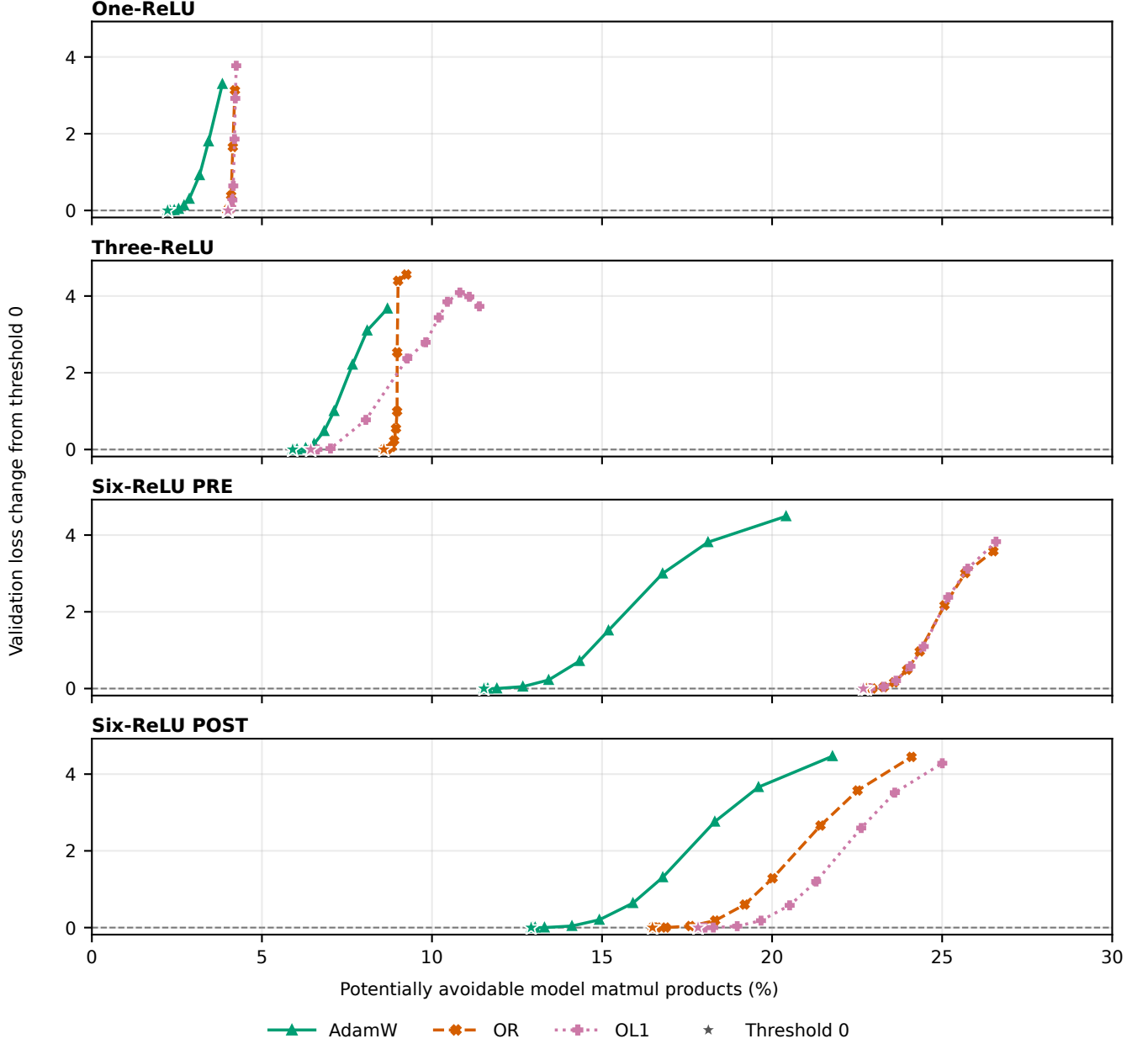


Exact zero means $x == 0$ with no tolerance. Lines connect measured thresholds.
 Logical zeros are not measured dense-kernel speedup.

Figure 11: Site-specific clipping frontiers with one architecture per row. All panels share axes; gray cells mark gates absent from an architecture. Exact-zero percentage is measured at the clipped site, and loss change is relative to that run's threshold-zero point.

Post-Hoc Clipping vs Potentially Avoidable Model Matmul Products

692,224 validation tokens per point; direct actual-operand counters.
Loss change is relative to threshold 0 within each eager-attention sweep.



Numerator: exact-zero logical products in QKV, valid-causal QK, valid-causal PV, W_O , W_1 , and W_2 .
Denominator adds the LM head; this is an opportunity proxy, not speedup.

Figure 12: Joint clipping versus potentially avoidable model matmul products. The numerator directly counts QKV, valid-causal QK, valid-causal PV, W_O , W_1 , and W_2 zero products; the denominator also includes the dense LM head. Stars mark threshold zero, whose x-coordinate is already nonzero when the trained ReLU gates create exact zeros.

Frontier readout. At the best tested joint threshold under a shared $\Delta\mathcal{L}_{\text{val}} \leq 0.05$ budget, R_{model} rises from 2.23 to 2.55% for One-ReLU AdamW, 5.90 to 6.28% for Three-ReLU AdamW, 11.53 to 11.91% for PRE AdamW, and 12.91 to 14.11% for POST AdamW. POST retains the clearest headroom across methods: OR rises from 16.48 to 17.60% and OL1 from 17.82 to 18.98%, whereas every non-POST gain is at most 0.60 points. PRE OR and OL1 both end near 23.28% because their Q/K gates are already nearly saturated. Site isolation favors nonuniform thresholds: at $t = .075$, POST V rises from 49–51% to 66–69% exact zeros within the same loss budget, while Q/K tolerate $t = .3$ with little loss. This supports per-site learned thresholds over one global cutoff.

7 Conclusions and Next Test

- **PRE is the stronger architecture control.** Relative to Three-ReLU AdamW, it almost doubles R_{model} with little additional loss. It is the cleanest evidence that post-QKV gating can open QK/PV sparsification paths.
- **POST is the stronger pressured control.** OR adds 3.57 percentage points of model-wide opportunity for +0.0366 loss; OL1 adds 4.91 points for +0.0740. Both avoid PRE pressure’s much larger loss cost in this one-seed test.
- **PRE pressure is too aggressive at the inherited setting.** OR/OL1 drive Q/K/V toward 90–100% zeros and roughly 22.7% model-wide opportunity, but lose about half a validation-loss point. This setting should be tuned, not declared ineffective.
- **Placement changes what the zero means.** PRE gates are upstream of RoPE and need pair-level survival accounting. POST gates act on the final QK operands. V is already a direct PV sparsity interface in both cases.
- **Dense timing answers a different question.** Gate and pressure overhead is visible, but no run measures sparse-kernel acceleration. Logical opportunity must be validated with an execution format that consumes it.
- **Scope is limited.** All comparisons are one seed, one model size, one token pass, and full-sequence training attention; they provide mechanism evidence, not uncertainty-qualified final estimates.

Recommended next steps, in order:

1. **Tune the Q/K/V budget.** Sweep smaller OR/OL1 weights and step budgets for PRE and POST, using R_{model} rather than mean gate sparsity as the target. Keep PRE AdamW and POST OR as fixed references.
2. **Separate QK from PV.** Run Q/K-only, V-only, and Q/K/V gates. This tests whether PRE’s loss comes from near-erasing attention scores, values, or both.
3. **Make PRE pair-aware.** Gate rotary coordinate pairs together so RoPE cannot repopulate an isolated scalar zero. Compare pair sparsity, QK products, and loss with the scalar PRE gate.
4. **Use clipping to initialize learnable thresholds.** Learn one nonnegative threshold per layer/site under a global product budget. Keep probability P as a later threshold-and-renormalize TODO rather than applying plain ReLU after softmax.
5. **Measure execution.** Implement or reuse zero-aware QK/PV and projection kernels, then report latency, memory traffic, and energy alongside logical products.
6. **Add uncertainty before scale-up.** Repeat the surviving controls across seeds at 14M, then retest rather than extrapolate their zero rates to larger Pythia models.

The immediate recommendation is therefore: **retain PRE AdamW as the architecture result and POST OR as the pressure result; next run a lower-pressure Q/K-only versus V-only matrix, with pair-aware PRE gating as the main architectural follow-up.**